



Instituto Superior de Engenharia

Politécnico de Coimbra

DEPARTAMENTO DE / DEPARTMENT OF
INFORMÁTICA E SISTEMAS

ReMed

Relatório de Licenciatura em / in Engenharia Biomédica

Autor / Author

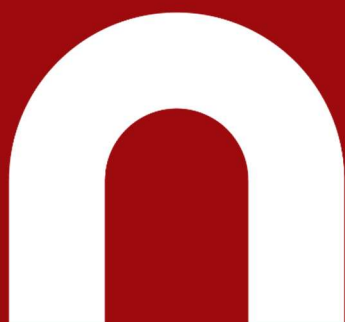
Mariana Pereira Costa

Nathalia Yumi Barbosa dos Santos Kuwae

Orientador

José Marinho

Coimbra, janeiro 2026



INSTITUTO POLITÉCNICO
DE COIMBRA

INSTITUTO SUPERIOR
DE ENGENHARIA
DE COIMBRA

Resumo

O envelhecimento populacional traz consigo a necessidade de ferramentas tecnológicas adaptadas à necessidade de ferramentas tecnológicas adaptadas que promovam a autonomia e qualidade de vida das pessoas mais idosas. No contexto da saúde, as aplicações móveis desempenham um papel vital ao mitigar dificuldades de memória e organização, facilitando a gestão de rotinas complexas que, de outra forma, poderiam comprometer o bem-estar dos utilizadores. Neste trabalho, irá ser analisado o desenvolvimento de uma aplicação móvel com o objetivo de auxiliar utilizadores na gestão diária dos seus medicamentos, garantindo uma maior adesão ao tratamento e promovendo a segurança de quem os toma.

Palavras-chave: Aplicação, Android Studio, Flutter, Firebase, Gerenciamento, Medicação

Abstract

Population aging brings with it the need for technological tools adapted to the specific needs of older adults, promoting autonomy and quality of life. In the healthcare context, mobile applications play a vital role in mitigating memory and organizational difficulties, facilitating the management of complex routines that could otherwise compromise users' well-being. In this work, the development of a mobile application aimed at assisting users in the daily management of their medications will be analyzed, ensuring greater adherence to treatment and promoting the safety of those who take them.

Keywords: Application, Android Studio, Flutter, Firebase, Management, Medications

Índice

Resumo	i
Abstract	ii
Índice	iii
Índice de figuras	v
1 Introdução	1
2 Requisitos da Aplicação	2
2.1 Requisitos Funcionais	2
2.2 Requisitos Não Funcionais	3
3 Metodologia	4
3.1 Tecnologias Utilizadas	4
3.2 Ferramentas e Pacotes	5
3.3 Processo de Desenvolvimento: Agile e Scrum	6
4 Arquitetura do Sistema	7
4.1 Estrutura geral do código	7
4.2 Componentes principais (interfaces, armazenamento, georreferenciação)	7
4.3 Modelo de dados	9
4.3.1 Coleção Users	9
4.3.2 Coleção Medicação	9
4.3.3 Coleção Farmácias	9
4.4 Fluxo de Dados na Aplicação	10
5 Implementação	11
5.1 Registo da autenticação dos utilizadores	11
5.2 Armazenamento local e em cloud	12
5.3 Integração com mapas e georreferenciação	15
6 Resultados	17

6.1	Comparação com objetivos iniciais	17
6.2	Limitações e desafios encontrados	18
7	Conclusão e Trabalhos Futuros	19
	Apêndice	22

Índice de figuras

3.1	Exemplos de alguns pacotes utilizados	6
5.1	Botão que permite ao utilizador escolher o tipo “Utente” e navegar para o ecrã seguinte	11
5.2	Barra de progresso linear que indica visualmente o avanço do utilizador no processo de registo	12
5.3	Verificação de campos obrigatórios, exibindo uma mensagem de erro caso o número de utente colocado não tenha 9 dígitos	12
5.4	Criação da conta Firebase usando email fictício e palavra-passe, retornando um ID único que identifica o utilizador	12
5.5	Armazena os dados básicos do utilizador no Firestore, vinculando-os ao UID retornado pelo Firebase Authentication	13
5.6	Função que procura os dados do utilizador no Firestore e atualiza o estado local do widget, garantindo sincronização entre cloud e interface	13
5.7	Atualiza localmente o idioma do app e persiste a escolha na cloud, permitindo sincronização entre sessões e dispositivos	13
5.8	Função que obtém e formata a data atual localmente, exibindo-a na interface sem depender de conexão à internet	14
5.9	Stream que obtém os medicamentos armazenados no Cloud Firestore e atualiza a interface em tempo real	14
5.10	Função responsável por construir visualmente cada medicamento da lista, aplicando lógica condicional com base no estado para diferenciar medicamentos pendentes dos já tomados	14
5.11	Atualização do estado do medicamento no Cloud Firestore quando o utilizador confirma a toma, assegurando que a alteração fica permanentemente guardada na cloud	14
5.12	Definição das coordenadas iniciais para centrar o mapa na cidade de Coimbra	15
5.13	Criação de um marcador de farmácia no mapa com interação de toque, exibindo detalhes ao utilizador	15
5.14	Recuperação em tempo real das farmácias do Firebase e conversão de suas coordenadas em marcadores no mapa	16
5.15	Renderização do mapa com tiles do OpenStreetMap e marcadores das farmácias, permitindo navegação e interação do utilizador	16

1 Introdução

Com o avançar da idade, a polifarmácia (toma de múltiplos medicamentos) torna-se comum, aumentando o risco de erros ou esquecimentos. Neste cenário, as tecnologias assistivas surgem como ferramentas essenciais que promovem a autonomia dos idosos. No entanto, para que uma solução digital seja eficaz para este público, não basta ser funcional. Deve ser acessível, combatendo as barreiras cognitivas e sensoriais que muitas vezes afastam os idosos da tecnologia.

O projeto em questão foi desenvolvido no âmbito da Unidade Curricular de Programação de Dispositivos Móveis, e consiste na criação de uma aplicação móvel com o objetivo de auxiliar utilizadores na gestão diária dos seus medicamentos, garantindo uma maior adesão ao tratamento e promovendo a segurança de quem os toma.

Além de enviar lembretes personalizados, a aplicação permite ao utilizador, a aplicação permite ao utilizador consultar farmácias próximas através de um mapa interativo e verificar onde é possível adquirir os seus medicamentos.

Inserida no tema ‘Saúde, Bem-Estar e Lazer’, a aplicação visa aumentar a autonomia de pessoas com doenças crónicas e idosos que necessitam de medicação com certa frequência. Permite ainda, que os cuidadores/responsáveis acompanhem o cumprimento do plano terapêutico em tempo real.

O desenvolvimento dessa aplicação foi antecedido por um protótipo em Figma, que serviu como identidade visual baseada em tons de azul pastel, de modo a reduzir o esforço cognitivo. Esta escolha foi feita tendo em conta os critérios dos fatores humanos e as heurísticas de usabilidade, de modo a não deixar a aplicação visualmente pesada ou complexa. Dado o carácter interativo do projeto, priorizou-se uma interface minimalista e organizada, uma vez que a solução será maioritariamente utilizada por idosos, que beneficiam de sistemas com menor carga de informação.

Por fim, o intuito é tornar a experiência de gestão mais simples, mais intuitiva e útil para o dia a dia dos utilizadores e dos seus respetivos responsáveis (caso haja). Ao longo deste documento iremos analisar de forma mais detalhada o desenvolvimento da aplicação em si.

2 Requisitos da Aplicação

A definição dos requisitos é um passo fundamental, uma vez que garante que a aplicação atenda às expectativas dos utilizadores e resolva os problemas relacionados à gestão da medicação. Neste contexto, foram estabelecidos requisitos que asseguram a autonomia do utente e a facilidade de supervisão por parte do cuidador ou responsável.

Contudo, importa salientar que, devido à delimitação temporal do projeto, optou-se — também por sugestão dos docentes — por não implementar a interface do responsável na aplicação final, ficando esta funcionalidade apenas explicitada no protótipo desenvolvido anteriormente.

2.1 Requisitos Funcionais

Os requisitos funcionais descrevem as ações que o sistema deve ser capaz de realizar. Para a aplicação desenvolvida, foram definidos os seguintes:

- Gestão do Plano de Medicação;
- Sistema de Alertas e Notificações;
- Modo Colaborativo (Responsável);
- Georreferenciação;
- Sincronização Multidispositivo.

A gestão do plano de medicação permite ao utilizador registar informações como o nome do medicamento, dosagem, frequência e duração de cada tratamento. Adicionalmente, foi desenvolvido um sistema de alertas que possibilita o envio de lembretes personalizados para a toma dos medicamentos no horário correto, permitindo ainda marcar o estado de cada toma como “*tomado*” ou “*pendente*”.

Foi também implementado um mapa interativo para a localização de farmácias próximas, facilitando a aquisição de medicamentos. Com o intuito de otimizar este processo, integrou-se uma funcionalidade de reconhecimento por câmara, que permite identificar medicamentos através das respetivas embalagens. Por fim, de forma a garantir a disponibilidade e integridade da informação, a aplicação suporta a sincronização multidispositivo através da *cloud*, possibilitando o acesso aos dados em qualquer plataforma.

2.2 Requisitos Não Funcionais

Os requisitos não funcionais referem-se às propriedades e restrições do sistema. Neste projeto, destacam-se os seguintes:

- Usabilidade e Acessibilidade;
- Segurança e Privacidade;
- Desempenho.

A interface da aplicação deve ser simples e intuitiva, recorrendo a fontes legíveis e contrastes adequados, de modo a facilitar a utilização por utilizadores com diferentes níveis de literacia digital. Os dados pessoais são protegidos, exigindo mecanismos de autenticação e autorização explícita para o acesso por terceiros. Adicionalmente, a aplicação deve apresentar um desempenho eficiente, sendo leve e responsiva, garantindo uma curva de aprendizagem reduzida e um uso eficaz sem exigir grande esforço por parte do utilizador.

3 Metodologia

Nesta secção, serão descritas as tecnologias e processos escolhidos para transformar o protótipo numa aplicação funcional, focando na eficiência do desenvolvimento e na praticidade na experiência do utilizador final.

3.1 Tecnologias Utilizadas

Para o desenvolvimento da aplicação, recorreu-se ao *Flutter*, um *framework* que permite a criação de aplicações móveis multiplataforma, nomeadamente para os sistemas Android e iOS, a partir de uma única base de código. Esta tecnologia facilita a construção de interfaces personalizadas, fluidas e responsivas, sendo particularmente relevante para a implementação de um design minimalista e intuitivo, adequado às necessidades de utilizadores mais idosos.

No Flutter, a interface é estruturada com base na arquitetura denominada *Árvore de Widgets*, na qual cada componente visual é representado por um widget. Os elementos não são independentes, mas organizados hierarquicamente, funcionando como “peças dentro de peças”. Para a disposição dos componentes na interface, foram utilizados *widgets* de múltiplos filhos, como **Column** e **Row**, permitindo uma organização clara e coerente dos elementos, bem como *widgets* de filho único, como o **Container**, usados para personalização do aspeto visual, incluindo margens, espaçamentos e cores.

A utilização de **StatefulWidget** na implementação da lista de medicamentos permite que a aplicação reaja de forma dinâmica às interações do utilizador. Sempre que o estado de uma toma é alterado, o método `setState()` notifica o Flutter para reconstruir a árvore de componentes, garantindo um feedback visual imediato, como a alteração da cor do estado de vermelho ou amarelo para verde. Por outro lado, o **StatelessWidget** é aplicado em componentes que não sofrem alterações de estado, tais como ícones ou rótulos estáticos.

A linguagem de programação utilizada pelo Flutter é o *Dart*, que se destaca pela sua execução rápida e eficiente. Esta linguagem disponibiliza bibliotecas que permitem a comunicação em tempo real com o *Firebase*, possibilitando que, sempre que o utente adiciona ou edita um medicamento, os dados sejam sincronizados instantaneamente com o *Cloud Firestore*, assegurando consistência e disponibilidade da informação.

Por fim, no desenvolvimento do projeto foram adotadas as metodologias *Agile* e *Scrum*, amplamente utilizadas no desenvolvimento de software por promoverem flexibilidade, adaptação a mudanças e entregas contínuas de valor. A abordagem Agile baseia-se em

princípios que privilegiam o desenvolvimento iterativo e incremental, a colaboração constante entre a equipa e os intervenientes do projeto, bem como a capacidade de resposta rápida a alterações nos requisitos. Em vez de um planeamento rígido e extensivo, esta abordagem propõe ciclos curtos de trabalho, permitindo feedback frequente e melhoria contínua do produto.

O *Scrum*, enquanto *framework* que operacionaliza os princípios Agile, fornece uma estrutura clara para a gestão e execução do projeto. O trabalho é organizado em iterações de duração fixa, denominadas *Sprints*, durante as quais são desenvolvidas funcionalidades previamente definidas e priorizadas. O Scrum define ainda papéis específicos, como o *Product Owner*, o *Scrum Master* e a equipa de desenvolvimento, bem como artefactos e eventos que asseguram transparência, inspeção e adaptação ao longo do ciclo de desenvolvimento, contribuindo para um maior controlo do progresso e para a qualidade do produto final.

3.2 Ferramentas e Pacotes

Para a concretização técnica do projeto, foi utilizado um conjunto de ferramentas e bibliotecas externas, fundamentais para a implementação das funcionalidades de georreferenciação, armazenamento na *cloud* e acessibilidade, anteriormente referidas.

Recorreu-se a ferramentas de suporte como o *Android Studio*, utilizado como ambiente de desenvolvimento integrado para a escrita do código em *Dart*; o *Firebase Console*, responsável pela configuração da base de dados e pela monitorização da autenticação dos utilizadores; e o *Figma*, que desempenhou um papel crucial na fase de prototipagem e design, garantindo que as heurísticas de usabilidade e os princípios de fatores humanos fossem devidamente considerados e cumpridos.

Adicionalmente, a capacidade do Flutter de dividir o sistema em componentes independentes, promovendo a modularidade, permite que diferentes partes da aplicação sejam desenvolvidas e testadas de forma isolada. Esta abordagem facilita a integração de pacotes específicos que simplificam o desenvolvimento do projeto. Entre os pacotes utilizados destacam-se o `firebase_auth`, responsável pela implementação do sistema de registo e autenticação, permitindo que o utilizador mantenha a sua sessão ativa; o `cloud_firestore`, uma biblioteca que fornece acesso à base de dados NoSQL, através da qual são geridos os medicamentos, horários e restantes informações relevantes; e o `flutter_map`, que possibilita a integração de um mapa interativo, permitindo ao utilizador localizar farmácias próximas de forma simples e intuitiva.

```
cloud_firestore: ^6.1.0
firebase_analytics: ^12.0.4
firebase_auth: ^6.1.2
flutter_map: ^8.2.2
latlong2: ^0.9.1
flutter_localizations:
  sdk: flutter
intl: ^0.20.2
provider: ^6.1.5+1
geolocator: ^14.0.2
```

Figura 3.1: Exemplos de alguns pacotes utilizados

3.3 Processo de Desenvolvimento: Agile e Scrum

No processo de desenvolvimento, foi essencial o seguimento dos princípios de Agile e Scrum.

O Agile caracteriza-se por uma mentalidade voltada à entrega contínua de partes funcionais do projeto, em vez de esperar pela conclusão tardia. No caso desta aplicação, o desenvolvimento ocorreu gradualmente ao longo das últimas aulas, nas quais novos desafios eram propostos semanalmente. Essa abordagem permitiu um progresso iterativo e possibilitou a concentração em diferentes componentes de forma segmentada.

O Scrum, por sua vez, é um método que se insere dentro do Agile, no qual o trabalho é dividido em pequenos ciclos chamados *sprints*. Essa divisão facilita o desenvolvimento da aplicação, pois permite focar em uma funcionalidade de cada vez — por exemplo, primeiro o login, depois o mapa, e assim por diante — garantindo que cada parte fosse testada e funcionasse corretamente.

4 Arquitetura do Sistema

A arquitetura do sistema define a organização e a interação dos seus diferentes componentes, garantindo a eficiência no processamento de dados. Nesta secção, apresenta-se a estrutura geral do código, destacando os principais módulos responsáveis pelas interfaces, armazenamento, autenticação e georreferenciação. Também é detalhado o modelo de dados utilizado, incluindo as coleções e a estrutura dos documentos no serviço Firestore, bem como o fluxo de dados na aplicação, evidenciando como as informações circulam entre o utilizador e a interface.

4.1 Estrutura geral do código

Relativamente à organização geral do código, os ficheiros `.dart` foram organizados numa pasta `lib/`, ou seja, o projeto foi organizado em diretórios que separam as várias interfaces. Recorreu-se ainda à arquitetura de widgets do Flutter, onde o ficheiro principal (`main.dart`) inicializa a aplicação e define o seu tema visual, enquanto os ecrãs específicos são construídos através do alinhamento de componentes `Stateful` e `StatelessWidgets`.

Adicionalmente, a utilização do `StreamBuilder` foi essencial para que a estrutura do código reagisse automaticamente a mudanças nos dados da cloud sem necessidade de alterações manuais, ou seja, a integridade dos dados foi garantida por uma camada de serviços que interagem com o Cloud Firestore em tempo real através de Streams.

4.2 Componentes principais (interfaces, armazenamento, autenticação, georreferenciação)

As interfaces da aplicação foram desenvolvidas com foco na usabilidade, acessibilidade e simplicidade, tendo em consideração o perfil do público-alvo, maioritariamente composto por pessoas idosas e utilizadores com menor literacia digital. Desde a fase de prototipagem até à implementação final, procurou-se minimizar a carga cognitiva, apresentando apenas a informação essencial em cada ecrã.

O design visual recorre a cores em tons de azul pastel, escolhida por transmitir tranquilidade e reduzir o esforço visual. A tipografia utilizada apresenta tamanhos adequados e bom contraste com o fundo, facilitando a leitura, enquanto os ícones funcionam como elementos de apoio ao texto, tornando a navegação mais intuitiva.

A estrutura de navegação foi concebida de forma linear e previsível, com ecrãs bem definidos e fluxos guiados. No processo de registo, por exemplo, são utilizadas barras de progresso que indicam claramente a etapa em que o utilizador se encontra, reduzindo a sensação de incerteza e aumentando a confiança na utilização da aplicação.

Foi também dada especial atenção à consistência da interface, mantendo padrões visuais e de interação semelhantes ao longo da aplicação, o que facilita a aprendizagem e utilização contínua. Sempre que possível, são fornecidos feedbacks visuais imediatos às ações do utilizador, contribuindo para uma experiência mais clara e acessível.

O processo de registo e autenticação dos utilizadores foi desenvolvido com o objetivo de ser simples, progressivo e adaptado a utilizadores com menos literacia digital, como os idosos. Para isso, utilizou-se várias fases para o registo, na qual a informação é recolhida de forma gradual.

A autenticação é feita através do Firebase Authentication que permite gerir identidades de forma segura sem necessidade de implementação de um servidor próprio. Neste trabalho, foi utilizado o método de autenticação por email e palavra-passe. Durante o processo de registo, o Firebase Authentication valida as credenciais fornecidas e cria automaticamente uma conta única, atribuindo-lhe um identificador exclusivo, id. Este identificador passa a representar o utilizador em todas as interações com os restantes serviços do Firebase.

O armazenamento de dados é realizado em cloud através do Firestore Database, uma base de dados NoSQL orientada a documentos. Os dados são organizados em coleções, sendo utilizada a coleção users para armazenar a informação associada a cada utilizador.

Cada utilizador possui um documento próprio, identificado pelo ID fornecido pelo Firebase Authentication, o que garante uma ligação direta entre a autenticação e os dados armazenados. Ao longo do processo de onboarding, os dados são guardados de forma progressiva no Firestore, permitindo atualizar campos específicos sem necessidade de reescrever todo o documento.

A interface da aplicação foi concebida tendo em atenção à simplicidade, acessibilidade e clareza, tendo em conta que o público-alvo inclui idosos. O design usa tons de azul, que reduzem o esforço visual. a tipografia utilizada apresenta tamanhos adequados e bom contraste, facilitando a leitura, enquanto os ícones funcionam como apoio visual ao texto.

A navegação foi estruturada de forma linear e previsível, com ecrãs bem definidos e fluxos guiados, como é o caso do processo de onboarding, que utiliza barras de progresso para indicar claramente o estado da tarefa. Esta abordagem permite ao utilizador compreender em que etapa se encontra, reduzindo a sensação de incerteza.

A georreferenciação baseia-se na utilização da localização atual do dispositivo móvel, mediante a autorização do utilizador, possibilitando a apresentação dos resultados de acordo com a sua posição geográfica. A informação é dada através de um mapa interativo, onde são apresentados os pontos de interesse, neste caso correspondem às farmácias nas

proximidades.

4.3 Modelo de dados

O modelo de dados da aplicação foi implementado utilizando o **Firestore Database**. Esta abordagem permite uma estrutura que facilita a leitura e a atualização dos dados em tempo real.

A informação é organizada em *coleções*, sendo cada coleção composta por vários *documentos*, identificados por um ID único. No contexto da aplicação, foram definidas três coleções principais: **Users**, **Medicação** e **Farmácias**.

4.3.1 Coleção Users

A coleção **Users** armazena os dados dos utilizadores registados na aplicação. Cada documento corresponde a um utilizador e é identificado pelo ID gerado automaticamente pelo Firebase Authentication.

A estrutura de um documento da coleção **Users** inclui os seguintes campos:

- nome_completo
- numero_utente
- email_fake
- data_registo

4.3.2 Coleção Medicação

A coleção **Medicação** contém informação sobre os medicamentos disponíveis na aplicação. Cada documento representa um medicamento específico (por exemplo, Paracetamol, Ibuprofeno ou Concor) e é identificado pelo nome do medicamento.

Os documentos desta coleção incluem campos como:

- Nome
- Unidade de Dosagem
- Dosagem – lista de valores possíveis de dosagem

4.3.3 Coleção Farmácias

A coleção **Farmácias** armazena informação relativa às farmácias disponíveis no sistema, sendo utilizada em conjunto com a funcionalidade de georreferenciação. Cada documento representa uma farmácia e contém dados de localização geográfica, com o campo principal sendo *localização*, que armazena as coordenadas geográficas.

4.4 Fluxo de Dados na Aplicação

O fluxo de dados na aplicação inicia-se quando o utilizador realiza o registo, fornecendo as suas credenciais e informações pessoais. Estes dados são validados pelo **Firestore** e armazenados no **Firestore**, vinculados ao UID do utilizador.

A partir daí, os dados relativos à medicação, horários e lembretes são inseridos pelo utilizador. Quando um utilizador ativa um lembrete de medicação, a aplicação consulta os dados armazenados, envia notificações no momento adequado e atualiza o estado do cumprimento do plano terapêutico em tempo real.

Paralelamente, se houver um cuidador associado, este consegue visualizar o progresso do utilizador através do **Firestore**, permitindo um acompanhamento remoto seguro e confiável.

A funcionalidade de georreferenciação integra-se neste fluxo, utilizando a localização atual do dispositivo para apresentar farmácias próximas, com os dados de cada estabelecimento também armazenados e atualizados na cloud.

Todo o fluxo de dados foi concebido para ser eficiente, seguro e transparente para o utilizador, mantendo a interface simples e intuitiva, de modo a facilitar a gestão diária da medicação sem sobrecarregar o utilizador com informações complexas.

5 Implementação

A implementação do sistema envolve a criação de funcionalidades centrais que garantem a interação eficiente do utilizador com a aplicação, bem como o armazenamento seguro de dados e a integração com serviços externos. Esta seção detalha os principais componentes da implementação, incluindo o registo e autenticação de utilizadores, estratégias de armazenamento local e em cloud, e a integração com mapas e georreferenciação. Além disso, são apresentados pedaços de código relevantes que exemplificam a forma como cada funcionalidade foi desenvolvida e integrada.

5.1 Registo da autenticação dos utilizadores

O registo inicia-se com a escolha do tipo de utilizador, apresentada de forma clara e intuitiva, utilizando botões com ícones que facilitam a identificação.

```
Expanded(
  child: ElevatedButton(
    onPressed: () {
      Navigator.push(
        context,
        MaterialPageRoute(builder: (context) => const NameInputScreen()),
      );
    },
    style: ElevatedButton.styleFrom(
      backgroundColor: const Color(0xFF3F61AF),
      padding: const EdgeInsets.symmetric(vertical: 20),
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(20),
      ),
    ),
    child: const Text(
      'UTENTE',
    ),
  ),
)
```

Figura 5.1: Botão que permite ao utilizador escolher o tipo “Utente” e navegar para o ecrã seguinte

Em seguida, o utilizador fornece informações pessoais, como o nome, o género e a data de nascimento. Cada etapa é acompanhada de barras de progresso, indicando a fase do registo, indicando a sensação de sobrecarga.

No ecrã de definir a palavra-passe, cada utilizador insere o seu N.º de Utente e a palavra-passe com validações que garantem campos preenchidos corretamente, palavra-passe mínima e números válidos.

Para integrar a autenticação com o Firebase, é criado um email fictício a partir do N.º de utente do utilizador. Esse email é usado para criar uma conta no Firebase Authentication

```
ClipRect(
  borderRadius: BorderRadius.circular(5.0),
  child: LinearProgressIndicator(
    value: 0.25,
    backgroundColor: const Color(0xFFB0D2F6),
    valueColor: AlwaysStoppedAnimation<Color>(_progressColor),
    minHeight: 10,
  ), LinearProgressIndicator
), ClipRect
```

Figura 5.2: Barra de progresso linear que indica visualmente o avanço do utilizador no processo de registo

```
if (userNumber.length != 9 || !RegExp(r'^[0-9]+$').hasMatch(userNumber)) {
  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(content: Text('Erro: O N° Utente deve ter exatamente 9 dígitos.')),
  );
  return;
```

Figura 5.3: Verificação de campos obrigatórios, exibindo uma mensagem de erro caso o número de utente colocado não tenha 9 dígitos

sem exigir um email real.

```
String fakeEmail = '$userNumber@remedapp.pt';

try {
  final userCredential = await _auth.createUserWithEmailAndPassword(
    email: fakeEmail,
    password: password,
  );

  final userUID = userCredential.user!.uid;
```

Figura 5.4: Criação da conta Firebase usando email fictício e palavra-passe, retornando um ID único que identifica o utilizador

Após a criação da conta, os dados do utilizador são armazenados no *Firestore*, garantindo a associação direta entre autenticação e dados pessoais. A coleção **users** armazena informações como **nome_completo**, **numero_utente**, **email_fake** e **data_registro**.

Desta forma, a implementação garante que a criação de contas e autenticação sejam seguras, guiadas e adaptadas ao público-alvo, utilizando recursos visuais claros, validações robustas e integração direta com o Firebase para persistência e gestão de dados.

5.2 Armazenamento local e em cloud

O armazenamento local e em cloud foi implementado para garantir que os dados do utilizador sejam sincronizados e acessíveis em diferentes dispositivos. Informações temporárias ou preferências do utilizador são armazenadas localmente, enquanto dados mais críticos, como o perfil, estão armazenados no Firebase.

```
await _firestore.collection('users').doc(userUID).set({
  'nome_completo': widget.userName,
  'numero_utente': userNumber,
  'email_fake': fakeEmail,
  'data_registro': FieldValue.serverTimestamp(),
});
```

Figura 5.5: Armazena os dados básicos do utilizador no Firestore, vinculando-os ao UID retornado pelo Firebase Authentication

No ecrã do perfil, por exemplo, os dados do utilizador são retirados diretamente da cloud, permitindo que o nome completo e o número de utente estejam sempre atualizados.

```
Future<void> _buscarDadosFirebase() async {
  final user = FirebaseAuth.instance.currentUser;
  if (user != null) {
    final doc = await FirebaseFirestore.instance.collection('users').doc(user.uid).get();
    if (doc.exists && mounted) {
      setState(() {
        _nomeCompleto = doc.data()?['nome_completo'] ?? "Sem nome";
        _numeroUtente = doc.data()?['numero_utente'] ?? "";
        _sexo = doc.data()?['sexo'] ?? "Pendente...";

        String? idiomaSalvo = doc.data()?['idioma'];
        if (idiomaSalvo != null) {
          _idiomaLocal = Locale(idiomaSalvo);
        }
      });
    }
  }
}
```

Figura 5.6: Função que procura os dados do utilizador no Firestore e atualiza o estado local do widget, garantindo sincronização entre cloud e interface

Para armazenar as preferências do utilizador, como o idioma selecionado, combinou-se o armazenamento local e cloud. O idioma é atualizado no dispositivo e enviado para o Firebase, garantindo que a escolha seja mantida mesmo ao trocar de dispositivo.

```
String novoCodigo = (n == 'English' ? 'en' : 'pt');
setState(() {
  _idiomaLocal = Locale(novoCodigo);
});

_guardarIdiomaNoFirebase(novoCodigo);
```

Figura 5.7: Atualiza localmente o idioma do app e persiste a escolha na cloud, permitindo sincronização entre sessões e dispositivos

No ecrã principal, dados temporários, como a data atual, são gerados e exibidos localmente, sem necessidade de consultar a cloud.

Este pedaço cria um fluxo de dados em tempo real a partir do Cloud Firestore, permitindo que a interface seja atualizada automaticamente sempre que um medicamento é adicionado, alterado ou removido na base de dados.

Este método cria o componente visual de cada medicamento, utilizando os dados vindos do Firestore e aplicando lógica condicional com base no estado (“Pendente” ou “Tomado”).

```
String _obterDataAtual() {
  final agora = DateTime.now();

  final formatador = DateFormat("EEEE, d 'de' MMMM 'de' y", "pt_PT");

  String dataFormatada = formatador.format(agora);

  return dataFormatada.substring(0, 1).toUpperCase() + dataFormatada.substring(1);
}
```

Figura 5.8: Função que obtém e formata a data atual localmente, exibindo-a na interface sem depender de conexão à internet

```
StreamBuilder<QuerySnapshot>({
  stream: FirebaseFirestore.instance.collection('Medicação').snapshots(),
  builder: (context, snapshot) {
    if (snapshot.hasError) return const Text('Erro ao carregar');
    if (snapshot.connectionState == ConnectionState.waiting) {
      return const Center(child: CircularProgressIndicator());
    }

    return Column(
      children: snapshot.data!.docs.map((doc) {
        Map<String, dynamic> data = doc.data() as Map<String, dynamic>;
        return _buildMedItem(
          doc.id,
          data['Nome'] ?? 'Sem nome',
          "${data['Dosagem'] ?? ''} • ${data['Horario'] ?? ''}",
          data['Status'] ?? 'Pendente',
        );
      }).toList(),
    );
  },
);
```

Figura 5.9: Stream que obtém os medicamentos armazenados no Cloud Firestore e atualiza a interface em tempo real

```
Widget _buildMedItem(String id, String nome, String info, String status) {
  bool isTomado = status.toLowerCase() == 'tomado';
```

Figura 5.10: Função responsável por construir visualmente cada medicamento da lista, aplicando lógica condicional com base no estado para diferenciar medicamentos pendentes dos já tomados

Quando o utilizador clica no botão “Tomei”, este trecho atualiza diretamente o documento no Firestore, alterando o estado do medicamento.

```
onTap: () => FirebaseFirestore.instance.collection('Medicação').doc(id).update({'Status': 'Tomado'}),
child: Container(
  padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 5),
  decoration: BoxDecoration(color: Colors.black, borderRadius: BorderRadius.circular(5)),
  child: const Text("Tomei", style: TextStyle(color: Colors.white, fontSize: 10)),
),
```

Figura 5.11: Atualização do estado do medicamento no Cloud Firestore quando o utilizador confirma a toma, assegurando que a alteração fica permanentemente guardada na cloud

5.3 Integração com mapas e georreferenciação

A integração com mapas e georreferenciação na aplicação permite localizar farmácias próximas ao utilizador e disponibilizar informações detalhadas sobre cada estabelecimento. Para isso, utilizou-se a biblioteca *flutter_map*, que possibilita a renderização de mapas interativos baseados em *OpenStreetMap*, e a cloud do *Firebase Firestore*, responsável por armazenar a localização georreferenciada das farmácias.

Definiu-se inicialmente a latitude e longitude de Coimbra como ponto central, garantindo que o mapa fosse exibido de forma centrada ao abrir a aplicação.

```
final double initialLat = 40.2033;
final double initialLng = -8.4103;
```

Figura 5.12: Definição das coordenadas iniciais para centrar o mapa na cidade de Coimbra

Cada farmácia é representada no mapa por um marcador interativo *Marker*, que ao ser tocado mostra um diálogo com informações detalhadas, incluindo o nome.

```
Marker _buildFarmaciaMarker(BuildContext context, LatLng point, String nome) {
  return Marker(
    point: point,
    width: 40,
    height: 40,
    child: GestureDetector(
      onTap: () => _showFarmaciaDetails(context, nome),
      child: const Icon(
        Icons.location_on,
        color: Colors.red,
        size: 40,
      ),
    ),
  );
}
```

Figura 5.13: Criação de um marcador de farmácia no mapa com interação de toque, exibindo detalhes ao utilizador

As farmácias são armazenadas numa coleção *Farmácias* da *Firebase*, onde cada documento contém um campo *Localização* do tipo *GeoPoint*. Os dados são acessados em tempo real por meio de um *StreamBuilder*, permitindo atualizações automáticas no mapa.

```

stream: FirebaseFirestore.instance.collection('Farmácias').snapshots(),
builder: (context, snapshot) {
  if (snapshot.hasError) {
    return Center(child: Text("Erro: ${snapshot.error}"));
  }
  if (snapshot.connectionState == ConnectionState.waiting) {
    return const Center(child: CircularProgressIndicator());
  }
  if (!snapshot.hasData || snapshot.data!.docs.isEmpty) {
    return const Center(child: Text("Nenhuma farmácia encontrada no Firebase."));
  }
  List<Marker> markers = snapshot.data!.docs.map((doc) {
    final data = doc.data() as Map<String, dynamic>;
    GeoPoint ponto = data['Localização'];
    double lat = ponto.latitude;
    double lng = ponto.longitude;
    String nomeFarmacia = doc.id;
    return _buildFarmaciaMarker(context, LatLng(lat, lng), nomeFarmacia);
  }).toList();
}

```

Figura 5.14: Recuperação em tempo real das farmácias do Firebase e conversão de suas coordenadas em marcadores no mapa

O mapa é renderizado utilizando o *FlutterMap*, com camadas (*TileLayer*) provenientes do *OpenStreetMap* e uma camada de marcadores (*MarkerLayer*). Essa abordagem garante que o mapa seja interativo e atualizado dinamicamente com os dados provenientes da cloud.

```

return FlutterMap(
  options: MapOptions(
    initialCenter: LatLng(initialLat, initialLng),
    initialZoom: 14.0,
  ),
  children: [
    TileLayer(
      urlTemplate: 'https://tile.openstreetmap.org/{z}/{x}/{y}.png',
      userAgentPackageName: 'pt.isec.pdm.firebase_1',
    ),
    MarkerLayer(markers: markers),
  ],
)

```

Figura 5.15: Renderização do mapa com tiles do OpenStreetMap e marcadores das farmácias, permitindo navegação e interação do utilizador

6 Resultados

6.1 Comparação com objetivos iniciais

O desenvolvimento da aplicação móvel foi precedido pela criação de um protótipo funcional em Figma, que teve como principal objetivo definir a identidade visual, a organização da informação e os fluxos de navegação da aplicação. Esta fase revelou-se fundamental para antecipar problemas de usabilidade e garantir que a solução final fosse adequada ao público-alvo, maioritariamente composto por idosos e pessoas com doenças crónicas.

De um modo geral, a aplicação desenvolvida no Android Studio manteve uma elevada fidelidade relativamente ao protótipo inicial. A estrutura das interfaces, a hierarquia da informação e os fluxos de navegação foram preservados, garantindo coerência entre o planeamento visual e a implementação técnica. As cores em tons de azul pastel, previamente definidas no protótipo, foram aplicadas na aplicação final, contribuindo para uma experiência visual mais calma e reduzindo o esforço cognitivo do utilizador. Alguns ícones, no entanto, apresentam ligeiras diferenças relativamente ao protótipo, tendo sido substituídos por alternativas disponíveis na biblioteca do Android Studio ou adaptadas para melhor legibilidade e compatibilidade com diferentes tamanhos de ecrã.

Relativamente à usabilidade, o protótipo permitiu testar antecipadamente a disposição dos elementos, como botões, campos de texto e ícones, assegurando que estes apresentassem dimensões adequadas e contraste suficiente para utilizadores com possíveis limitações visuais. Na aplicação final, estes princípios foram mantidos, embora tenham sido necessárias pequenas adaptações devido a limitações técnicas do Android Studio e à necessidade de garantir compatibilidade com diferentes tamanhos de ecrã e resoluções.

No que diz respeito às funcionalidades, o protótipo apresentava uma simulação das principais ações da aplicação, nomeadamente o registo de medicamentos, a configuração de lembretes, a consulta de farmácias próximas através de um mapa e a associação de cuidadores aos utentes. Estas simulações permitem explorar de forma conceitual a navegação entre telas e a interação com os elementos da interface, mas não envolviam funcionalidades reais nem integração com dados externos. Na implementação final, as funcionalidades de registo de medicamentos e consulta de farmácias tornaram-se totalmente funcionais, permitindo ao utilizador interagir efetivamente com dados reais e mapas interativos. Algumas funcionalidades previstas, como as notificações de lembretes e a funcionalidade de acompanhamento por cuidadores, não foram implementadas nesta fase. Estas limitações evidenciam diferenças em relação ao protótipo, mas não comprometem a estrutura

geral da aplicação, que manteve coerência com o planeamento inicial, a hierarquia de informação e os objetivos centrais do projeto.

Importa ainda referir que, durante o desenvolvimento, foram introduzidos pequenos ajustes face ao protótipo inicial, com o objetivo de melhorar a experiência do utilizador. Alguns elementos visuais foram simplificados e certas interações foram tornadas mais diretas, reduzindo o número de passos necessários para executar tarefas frequentes.

6.2 Limitações e desafios encontrados

Durante o desenvolvimento da aplicação, foram identificadas algumas limitações e desafios que influenciaram a implementação das funcionalidades previstas. Uma das principais limitações foi a não implementação da funcionalidade de responsáveis/cuidador. Embora o protótipo contemplasse a possibilidade de associar um cuidador a cada utente, permitindo monitorizar o cumprimento do plano terapêutico em tempo real, esta funcionalidade não foi concretizada devido a restrições de tempo e complexidade na gestão de permissões e sincronização de dados entre utente e cuidador.

Outro desafio esteve relacionado com a implementação dos dados no Firebase. Inicialmente, a aplicação foi projetada para armazenar diversas informações, incluindo género, data de nascimento e detalhes completos dos medicamentos, diretamente na base de dados. No entanto, nem todos os campos foram integrados na versão final, resultando em dados parcialmente armazenados ou geridos apenas localmente.

Uma outra limitação relevante prende-se com a não implementação da funcionalidade de reconhecimento de medicamentos através da câmara. Embora esta funcionalidade estivesse prevista nos requisitos iniciais e tivesse como objetivo facilitar a identificação automática dos medicamentos a partir das suas embalagens, a sua concretização revelou-se tecnicamente mais complexa do que o esperado.

Além destas questões, surgiram desafios técnicos relacionados com a adaptação do protótipo do Figma para a implementação em Android Studio, especialmente no que diz respeito à compatibilidade de ícones, dimensões de ecrã e fidelidade visual da interface.

Apesar destas limitações, a aplicação conseguiu cumprir os objetivos centrais do projeto, permitindo aos utilizadores registar medicamentos e consultar farmácias próximas, mantendo uma interface acessível e intuitiva.

7 Conclusão e Trabalhos Futuros

O presente trabalho teve como objetivo o desenvolvimento de uma aplicação móvel orientada para a gestão diária da medicação, direcionada principalmente a pessoas idosas e a utilizadores com doenças crónicas. A motivação central do projeto surgiu da necessidade crescente de soluções tecnológicas acessíveis que promovam a autonomia, reduzam erros associados à polifarmácia e contribuam para a melhoria da qualidade de vida dos utilizadores.

Ao longo do desenvolvimento, foi possível transformar um protótipo concebido em Figma numa aplicação funcional, recorrendo ao Flutter e aos serviços disponibilizados pelo Firebase. A aplicação permite o registo e autenticação de utilizadores, o armazenamento seguro de dados na cloud, a consulta de farmácias próximas através de um mapa interativo e a gestão básica da informação relacionada com a medicação. Todo o processo foi orientado por princípios de usabilidade e acessibilidade, tendo em conta as limitações cognitivas e sensoriais do público-alvo, através de uma interface simples, organizada e visualmente confortável.

No entanto, o projeto apresenta ainda margem significativa para melhorias e expansões futuras. Uma das principais evoluções passa pela implementação completa do modo colaborativo, permitindo a associação efetiva de cuidadores ou responsáveis aos utentes, com acesso ao estado das tomas e notificações em tempo real. A integração de um sistema de notificações push para lembretes de medicação constitui igualmente uma melhoria essencial para aumentar a adesão ao tratamento.

Adicionalmente, poderão ser exploradas funcionalidades como o reconhecimento automático de medicamentos através da câmara, a inclusão de relatórios de histórico de tomas, a personalização mais avançada da interface (por exemplo, tamanho de letra e modos de alto contraste) e a expansão da aplicação para suportar diferentes idiomas. A realização de testes de usabilidade com utilizadores reais, especialmente idosos, permitiria ainda recolher feedback valioso para otimizar a experiência de utilização.

Em síntese, este projeto demonstrou a relevância das aplicações móveis como ferramentas de apoio à saúde e evidenciou o potencial da tecnologia para responder a desafios reais do quotidiano. As bases desenvolvidas ao longo deste trabalho constituem um ponto de partida sólido para futuras iterações e melhorias, com vista à criação de uma solução mais completa, robusta e centrada no utilizador.

Bibliografia

- [1] MARINHO, José Manuel Meireles. *Diapositivos da disciplina de Flutter e Dart*, 2025, ISEC.



**Instituto Superior
de Engenharia**

Politécnico de Coimbra

Apêndice - Relatório Protótipo

1. Introdução

No âmbito da Unidade Curricular de Programação de Dispositivos Móveis, foi-nos proposto o desenvolvimento de uma aplicação seguindo alguns critérios previamente impostos. Com o intuito de cumprí-los, o protótipo criado (em Figma) tem como objetivo principal auxiliar os utilizadores (principalmente idosos) na gestão e controlo da toma de medicamentos de forma simples e prática, com a possibilidade de adicionar uma pessoa responsável (seja da família ou algum cuidador específico) para auxiliar na gestão dos medicamentos.

Nosso protótipo está dividido em três interfaces distintas. Uma relacionada ao utente que já possui uma conta, já tem medicamentos na sua lista e recebe a notificação para permitir o acesso de uma outra pessoa (responsável) ; outra relacionada ao utente que vai criar a sua conta pela primeira vez; e por fim, uma outra do ponto de vista do responsável.

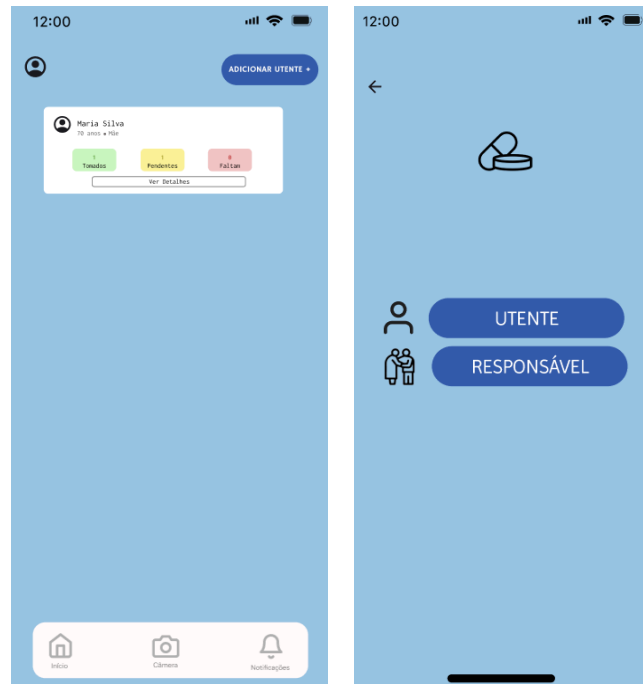
2. Inspiração para o desenvolvimento da interface

O desenvolvimento do protótipo teve como ponto de partida e inspirações, aplicações já existentes para o controlo de medicação. Contudo, procurou-se desenvolver uma versão mais acessível, funcional e com um design mais simplificado.

Para isso, optamos pelo desenvolvimento de um design limpo e de estética agradável aos utilizadores, sem muitos passos adicionais para chegar ao objetivo principal da aplicação. Para além disso, preocupamo-nos em elaborar um protótipo funcional que respondesse às necessidades do utilizador sem exigir grande esforço de aprendizagem.

Como mencionado anteriormente, nosso público-alvo é constituído principalmente, embora não exclusivamente, por pessoas idosas (acima dos 65 anos), uma vez que pertencem a um grupo de utilizadores que necessita de manter a sua medicação atualizada e organizada. Para além das inspirações em aplicações anteriores, inspirámo-nos também em pessoas com quem convivemos diariamente, que têm medicação diária e horários a cumprir. De forma a auxiliá-las a não se esquecerem e a conseguirem organizar os horários em que necessitam, de tomar os seus medicamentos, chegámos à ideia deste projeto.

Adicionalmente, criámos uma outra vertente associada à possibilidade de outras pessoas, nomeadamente responsáveis ou cuidadores, terem também acesso e a capacidade de adicionar medicamentos e ajudar os utilizadores a não se esquecerem das suas medicações, podendo ainda consultar quais foram tomadas, quais estão pendentes e quais ainda terão de ser realizadas nos respetivos horários.



3. Caracterização dos Utilizadores e Fatores Humanos

Como já referido no ponto anterior, o nosso público-alvo é composto maioritariamente por pessoas idosas acima dos 65 anos. Para esse efeito, procedemos à criação de um protótipo de simples e fácil utilização, e rápida aprendizagem, já que os nossos destinatários principais geralmente não estão tão familiarizados com as tecnologias atuais.

Preocupamo-nos em definir instruções claras e intuitivas, desde o preenchimento das informações necessárias para a criação da conta, até à adição e gestão dos medicamentos, com a combinação de um conjunto de ícones de sinalização e cores apropriados para cada situação.

Os fatores humanos focam-se em conceber sistemas adequados às necessidades dos utilizadores, e em como as características, capacidades e limitações dos utilizadores influenciam o design do sistema. Ou seja, o protótipo deve-se adaptar às necessidades humanas de modo a ser seguro, eficiente e fácil de usar. Engloba alguns pontos, como:

1. Facilidade de utilização:

Proporciona uma interface intuitiva e simples, sem muita informação e menus complexos com excesso de opções, de modo a não complicar a utilização, mas sim facilitar e contribuir para uma melhor gestão de medicamentos na vida cotidiana dos utilizadores.

2. Capacidade cognitiva:

Considerando que pessoas idosas podem ter uma memória mais limitada e dificuldades em processar várias informações só de uma vez, recorremos a instruções e lembretes claros, facilitando, portanto, a identificação rápida da informação.

3. Perceção visual e auditiva:

O protótipo contém textos com boa legibilidade, utilizando contrastes e cores adequadas a cada contexto. As notificações são exibidas no ecrã do smartphone através de alertas visuais e acompanhadas por sinais sonoros, caso o volume esteja ativado. Além disso, o sistema gera mensagens internas sempre que um medicamento é adicionado ou removido, garantindo que o utilizador esteja sempre informado sobre qualquer alteração.

4. Motivação e comportamento:

Os utilizadores, especialmente os idosos, podem demonstrar menos paciência e motivação para utilizar aplicações e acompanhar a evolução tecnológica. Para esse efeito, foram incorporados elementos de feedback positivo, que confirmam a conclusão das tarefas e tornam a interação mais simples, com intuito de incentivar o uso.

5. Segurança e confiança:

Os dados do utilizador são protegidos de modo a só ele ter acesso através da sua conta. Para além disso, o responsável (caso haja) terá de aguardar que o pedido de acesso a conta do utente seja aceite pelo mesmo, para só assim, ter acesso aos seus medicamentos.

4. Design, Cores, Objetos de Interação e Composição da Interface

Relativamente ao design, optamos por utilizar cores e objetos de interação que fossem intuitivos.

A atribuição incoerente de cores dificulta a aprendizagem. Se bem utilizada, orienta a visão para a informação, tornando agradável e eficaz a sua retenção. Para esse efeito, recorremos a tons de azul pastel como cor padrão do protótipo, uma vez que o olho humano é menos sensível aos comprimentos de onda azul.



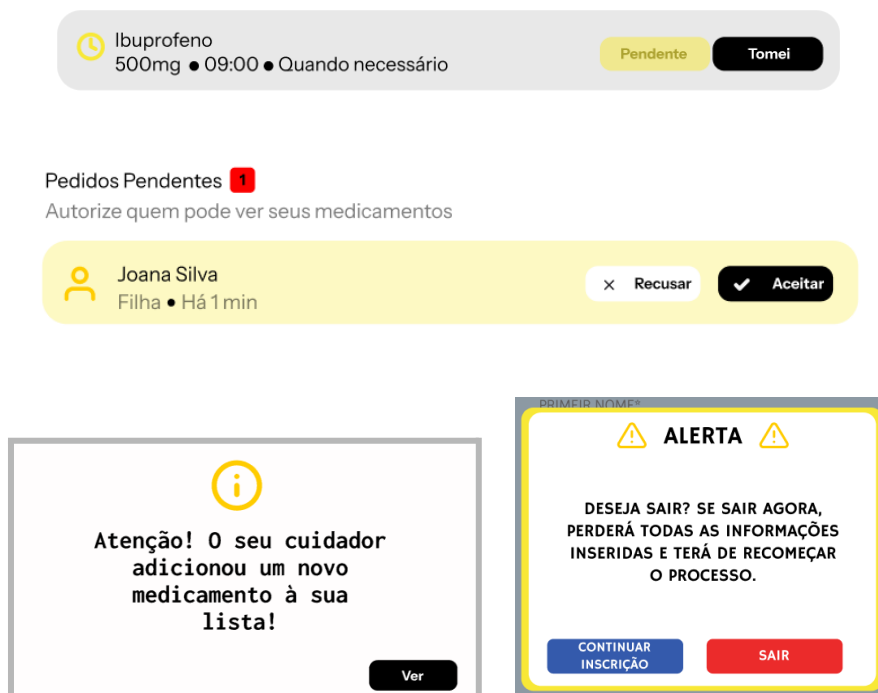
Utilizamos um tom pastel para o fundo e tons de azul mais escuros para os botões, de modo a deixar claro que se tratam de botões e facilitar a percepção da sua localização. A cor preta foi utilizada nos ícones, de forma a sobressair sobre o fundo azul e, assim, destacá-los visualmente.

Além disso, recorremos à cor branca para evidenciar as caixas de texto, nomeadamente as notificações internas do protótipo e a lista de medicamentos, por exemplo. O branco tem um brilho

característico e intenso que pode causar algum desconforto visual quando observado por um período prolongado. No entanto, não considerámos que os utilizadores permanecerão tempo suficiente na aplicação para que isso se torne um problema, uma vez que o seu uso se limita essencialmente às ações de verificar, adicionar, eliminar ou marcar medicamentos como tomados.

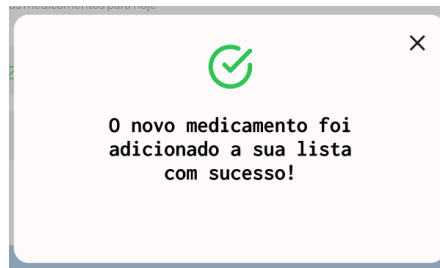


O amarelo foi utilizado com o intuito de chamar atenção e alertar o utilizador, por exemplo, para algum medicamento pendente ou para pedidos por parte dos cuidadores que ainda aguardam resposta.

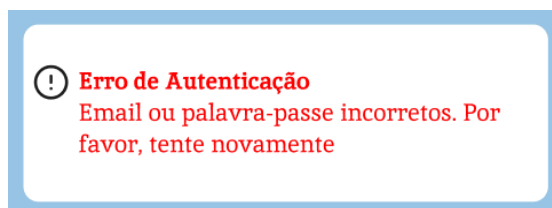


O verde foi reservado para confirmar ações e indicar medicamentos já tomados, funcionando como sinal positivo e de sucesso nas interações.





Por fim, o vermelho foi reservado para mensagens de erro ou para interromper uma ação, como interromper uma inscrição ou terminar a sessão.



Os ícons escolhidos seguem a mesma intuição das cores, mantendo uma linguagem visual coerente e facilmente compreensível. Cada um foi selecionado de forma a transmitir a sua função de maneira intuitiva, facilitando a identificação das ações na aplicação.

5. Usabilidade e Acessibilidade

As heurísticas de Nielsen são princípios reconhecidos no design de interfaces que ajudam a garantir que um sistema seja fácil, intuitivo e agradável de utilizar. A utilização das heurísticas é importante, pois estas oferecem critérios práticos para avaliar a experiência do utilizador, permitindo identificar falhas e propor melhorias nas interfaces.

Estas heurísticas são especialmente relevantes em aplicações destinadas a utilizadores idosos ou com pouca experiência tecnológica, como este projeto de gestão de medicamentos. Elas têm como objetivo orientar o design para que a interface seja clara, consistente, previsível e segura, reduzindo o risco de erros e aumentando a confiança do utilizador.

A primeira heurística, denominada Visibilidade do Estado do Sistema, destaca a importância de manter o utilizador constantemente informado sobre o que acontece durante a interação. Logo, a interface tem de assegurar que o utilizador compreenda a sua posição dentro do sistema, mostrando de forma clara em que ecrã se encontra, quais ações foram executadas e quais são os próximos passos possíveis. Esta visibilidade contínua cria um sentimento de controlo.

Neste protótipo, a heurística é respeitada através da presença das mensagens visuais que mostram o estado de cada medicamento, como as etiquetas coloridas que mostram se o medicamento já foi

tomado, se está pendente ou em falta. O botão de confirmação muda de cor quando a ação é

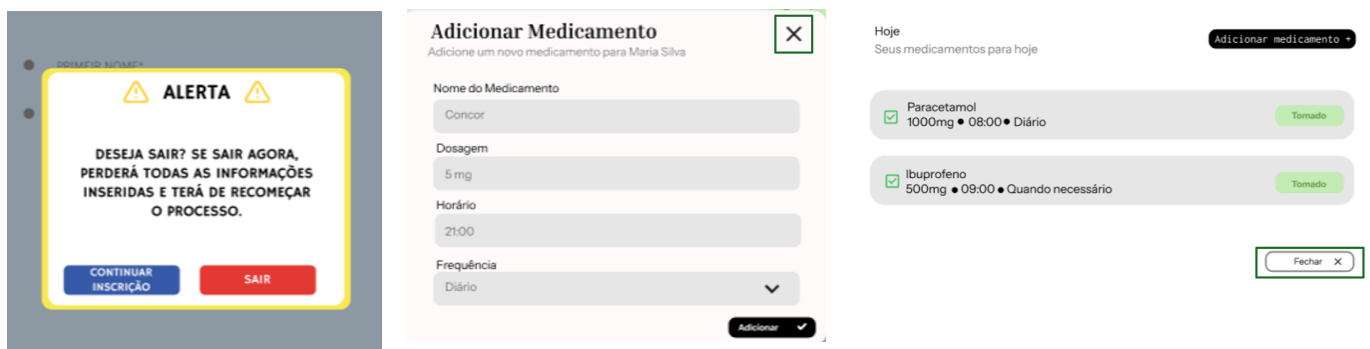


concluída, transmitindo ao utilizador a sensação de controlo.

A segunda heurística, compatibilidade entre o sistema e o mundo real, a linguagem utilizada deve ser familiar e de fácil compreensão para o utilizador, refletindo o modo como a sociedade pensa e comunica no dia a dia. No nosso protótipo, essa compatibilidade é garantida pela utilização de ícones universais, como o símbolo do comprimido e do perfil e pela adoção de uma linguagem simples, evitando termos técnicos, como “Adicionar Medicamento” ou “Cuidador”. Estas expressões fazem parte do vocabulário comum dos utilizadores idosos.



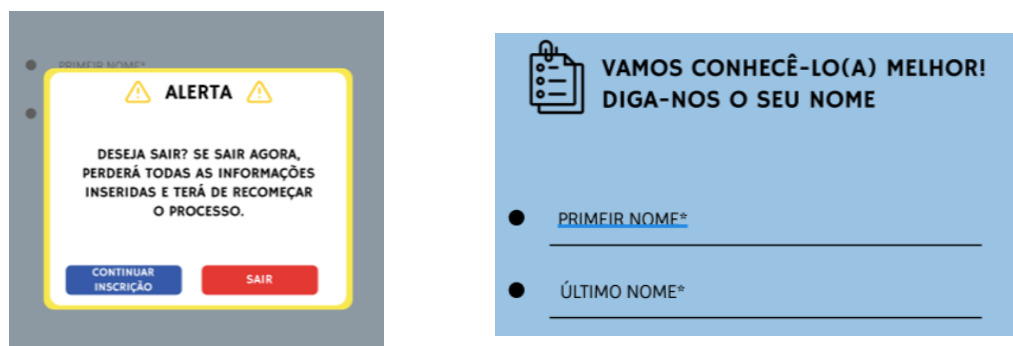
A terceira heurística, controlo e liberdade do utilizador, está diretamente relacionada com o utilizador poder desfazer ações ou navegar livremente sem o medo de cometer erros irreversíveis. Os botões de “Voltar” e as mensagens de confirmação antes de sair de determinadas páginas, como do processo de registo, permite essa liberdade. Desta forma, os utilizadores sentem que conseguem corrigir um erro facilmente e que mantêm o controlo sobre as suas ações.



A consistência e padrões, a quarta heurística, reforça a importância de manter a coerência em toda a interface. O protótipo segue um esquema de cores, onde predomina a cor azul e branco, que reduzem o cansaço visual, facilitam a leitura e tornam a navegação mais agradável. Os ícones têm o mesmo estilo gráfico e os botões de ação seguem sempre o mesmo formato e posição nos diferentes ecrãs, o que faz com que o utilizador antecipe o funcionamento da app.

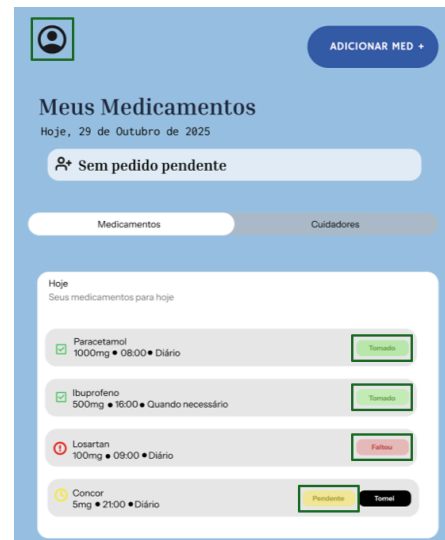


A prevenção de erros que faz a quinta heurística, sugere que a interface seja desenhada de modo a evitar que o utilizador cometa enganos, em vez de apresentar mensagens de erros depois do engano acontecer. Nesta aplicação, a heurística é representada através de campos de preenchimento obrigatório e da confirmação antes das ações mais críticas, como a **eliminação de medicamentos** ou a saída do registo.



A sexta heurística, reconhecimento em vez de memorização, incentiva o design de interfaces que permite ao utilizador reconhecer opções e comandos em vez de depender da memória. Esta

heurística pode ser encontrada no projeto através de ícones familiares e etiquetas textuais em todos os botões e secções. A lista de medicamentos apresenta também informações visuais, como a cor

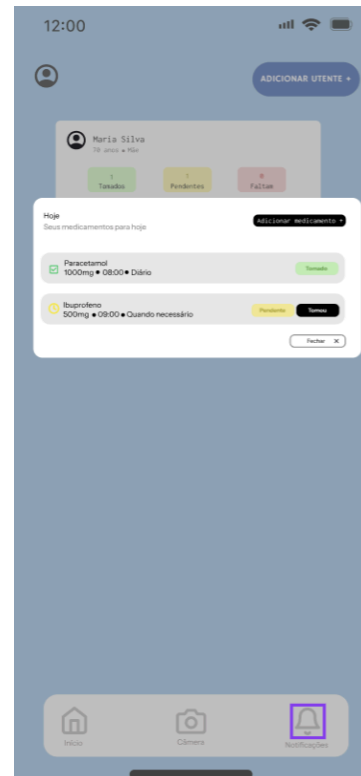


que corresponde ao estado do medicamento.

A flexibilidade e eficiência de uso, a sétima heurística, refere-se à capacidade do sistema adaptar-se a diferentes tipos de utilizadores, principiantes e experientes. No caso da aplicação desenvolvida, a interface foi pensada para ser intuitiva e acessível a utilizadores idosos, mas também oferece funcionalidades adicionais para os responsáveis que pretendam gerir mais do que um perfil de utente. Esta flexibilidade permite que a aplicação se ajuste às diferentes necessidades de quem a utilize.



O design estético e minimalista, que equivale à oitava heurística, refere que o sistema deve apresentar apenas informações relevantes e necessárias, evitando elementos desnecessários que possam distrair ou confundir o utilizador. No nosso protótipo, a heurística é aplicada através de um design simples, com espaços amplos, tipografia legível e uso equilibrado de cores suaves. O que oferece uma interface limpa e organizada, que facilita a leitura e reduz o esforço cognitivo.



A nona heurística, ajuda os utilizadores a reconhecer, diagnosticar e recuperar de erros, destaca a importância de o sistema fornecer mensagens de erro compreensíveis e orientar o utilizador na sua resolução. No protótipo, sempre que ocorre uma situação potencialmente problemática, surge uma mensagem clara com ícones visuais de aviso e opções de ação, como “Tentar novamente” ou “Cancelar”. Isto permite que o utilizador compreenda facilmente o que aconteceu e como pode corrigir o problema.

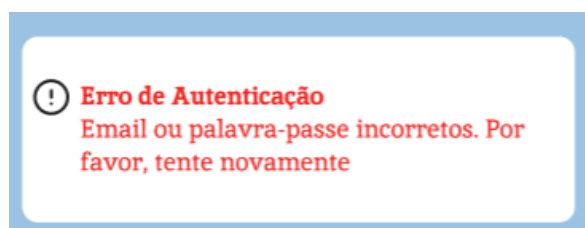
Adicionar Novo Utente

Conecte-se a um utilizador já existente

Número De Utente

O utente receberá um pedido de autorização.

Parentesco



Por fim, a décima heurística, ajuda e documentação, refere-se à existência de suporte acessível ao utilizador caso este precise de orientação adicional. Embora o protótipo tenha sido desenvolvido para ser o mais intuitivo possível, está prevista uma área de ajuda no perfil, onde o utilizador pode

consultar instruções simples ou contactar um cuidador responsável. Este recurso reforça a acessibilidade da aplicação e contribui para uma experiência de utilização mais segura e confiável.



6. Conclusão

O protótipo desenvolvido demonstra um desempenho sólido no cumprimento do seu objetivo principal: facilitar a gestão de medicamentos, especialmente para utilizadores idosos, através de uma interface simples e intuitiva. A coerência no uso das cores, ícones e elementos visuais contribui para uma navegação fluida e uma experiência de utilização positiva, reforçando a acessibilidade e a facilidade de aprendizagem.

Entre os pontos fortes destacam-se a clareza do design, a coerência visual, a adequação das heurísticas e o cuidado na adaptação do sistema às limitações cognitivas e sensoriais do público-alvo. Estes aspetos tornam o protótipo intuitivo e funcional, promovendo a autonomia e segurança do utilizador.

LINK para o nosso protótipo:

<https://www.figma.com/proto/yNpYGzFwuCCKr2pOXp8OI4/ReMed?node-id=0-1&t=jYrAl7O2FGC0635m-1>